



preparatic 31

TEMA 126

LA SEGURIDAD EN EL NIVEL DE APLICACIÓN.
TIPOS DE ATAQUES Y PROTECCIÓN DE
SERVICIOS WEB, BASES DE DATOS E
INTERFACES DE USUARIO.

Versión	31.1
Fecha de actualización	21/02/2026

Tema

ÍNDICE

1	La seguridad a nivel de aplicación	3
2	Seguridad en interfaces de usuario	3
2.1	Autenticación en aplicaciones o servicios	3
2.2	Autorización en el sistema	3
2.3	Gestión de excepciones	3
2.4	Validación de datos de entrada	3
2.5	Trazabilidad y auditorías	4
3	Seguridad en aplicaciones web	4
3.1	OWASP (Open Web Application Security Project)	4
3.2	Amenazas y ataques en aplicaciones web	4
3.2.1	A01:2025 Pérdida de control de acceso	4
3.2.2	A02:2025 Configuración de seguridad incorrecta	4
3.2.3	A03:2025 Uso de componentes con vulnerabilidades conocidas	5
3.2.4	A04:2025 Exposición de datos sensibles	6
3.2.5	A05:2025 Inyección	6
3.2.6	A06:2025 Diseño inseguro	7
3.2.7	A07:2025 Pérdida de autenticación	8
3.2.8	A08:2025 Software or Data Integrity Failures	9
3.2.9	A09:2025 Registro, monitorización y alerta insuficientes	9
3.2.10	A10:2025 Manejo incorrecto de condiciones excepcionales	10
3.3	OWASP cuadro resumen	10
3.3.1	Otros ataques	11
3.4	Metodologías de seguridad en aplicaciones web	12
3.5	La seguridad desde el punto de vista de la arquitectura de sistemas	12
4	Seguridad en servicios web	13
	Principales riesgos en seguridad de API	13
4.1	API REST	15
4.2	API SOAP	15
4.3	WS-Security	16
4.3.1	Seguridad a nivel de transporte (TLS) y a nivel de mensaje (WSS)	16
4.3.2	Tecnologías de autenticación, confidencialidad e integridad soportadas por WSS ..	17
4.3.3	Ejemplo de extensiones WSS	17
4.3.4	Otras extensiones para web services	18
4.3.5	Crítica a WSS y alternativas	20
4.4	SAML	20
	Características generales	21

	Mecanismos de funcionamiento	21
	SAML y Web Service-Security	24
4.5	JSON Web Token	25
	Casos de uso	25
	Estructura	25
	Funcionamiento	26
5	Seguridad de las bases de datos	28

1 La seguridad a nivel de aplicación

La implementación de aplicaciones, cada vez más orientadas a sistemas abiertos, hace más necesario que, además de tener en cuenta cuestiones de seguridad a nivel de infraestructura o comunicaciones, sea necesario tener la seguridad en mente en el desarrollo de aplicaciones desde el comienzo de los proyectos.

Esta gestión debe ser multidisciplinar en cada una de las capas implicadas. En este tema resumimos las implicaciones en cuanto a la interfaz de usuario, la protección de los servicios web o la base de datos.

2 Seguridad en interfaces de usuario

La interfaz de usuario es el elemento de interacción entre el usuario y el sistema, y es preciso contemplar cuestiones de seguridad en su definición. Nótese la diferencia entre una interfaz de usuario, que abarca la interacción entre un usuario humano con el sistema de información, a API (Application Programming Interface) que hace referencia a la interacción entre sistemas o aplicaciones.

2.1 Autenticación en aplicaciones o servicios

Autenticación es el proceso por el que el usuario acredita su identidad a través de unas credenciales (algo que se tiene, como una tarjeta, algo que se sabe, como una contraseña, o algo que se es, como un elemento biométrico). Por lo general, se considera segura una combinación de dos de estos elementos, por ejemplo, una tarjeta criptográfica combinado con un elemento biométrico como la huella dactilar.

Las posibles amenazas en este elemento tienen que ver con el robo de identidad de una persona, es decir, identificarte ante un sistema como quien no eres, o en la suplantación de una sesión activa de otro usuario.

2.2 Autorización en el sistema

La autorización es el proceso por el que el sistema comprueba que un usuario tiene permiso para realizar la operación que ha solicitado. Las dos amenazas a evitar son la concesión de permisos superiores a los que debe tener un usuario, así como la denegación de permisos a un usuario que deba tenerlos. La gestión de autorizaciones se suele implementar mediante distintos mecanismos como las listas de control (ACL), o los modelos de control basados en roles (RBAC), o atributos (ABAC).

2.3 Gestión de excepciones

Las excepciones son comportamientos no esperados de los sistemas. En el caso de la interfaz de usuario, ante una excepción en la operativa, es conveniente que este hecho no proporcione al usuario información interna del sistema, que un usuario malintencionado pueda aprovechar para atacar al mismo.

2.4 Validación de datos de entrada

Un elemento crítico es la validación de datos proporcionados por el usuario a la interfaz de usuario. Es un elemento que suele obviarse en una implementación deficiente de un sistema y que puede provocar que, a través de la introducción de datos inválidos, un atacante interactúe con un sistema de forma no deseada, con objeto de obtener información a la que no está autorizado, suplantar a otro usuario, uso de inyección SQL, ataques de denegación de servicio, etc.

2.5 Trazabilidad y auditorías

Para la identificación de operativas incorrectas es conveniente que todas las capas de un sistema guarden información de excepciones o errores que permitan trazar su origen y, en su caso, corregir posibles defectos del software.

3 Seguridad en aplicaciones web

3.1 OWASP (Open Web Application Security Project)

La Fundación OWASP es un organismo sin ánimo de lucro formado por empresas, organizaciones educativas y particulares de todo el mundo. Es una comunidad de seguridad informática que trabaja para crear artículos, metodologías, documentación y herramientas que se liberan y pueden ser usadas gratuitamente. Publica y actualiza cada 3 años el proyecto **OWASP Top 10** que es un documento donde se recogen los diez riesgos de seguridad más importantes en aplicaciones Web. Mientras que **OWASP Top 10** indica que evitar, el **ASVS (Application Security Verification Standard)** es una lista de verificación detallada para auditar y asegurar que una aplicación Web es realmente robusta.

3.2 Amenazas y ataques en aplicaciones web

Los diez riesgos más críticos en Aplicaciones Web según OWASP Top 10 - 2025 son:

3.2.1 A01:2025 Pérdida de control de acceso

Si no se aplican correctamente las restricciones sobre lo que los usuarios pueden hacer, un atacante puede acceder de forma no autorizada a funcionalidades y datos, cuentas de otros usuarios, ver archivos sensibles, modificar datos, cambiar permisos de acceso, etc.

Cómo prevenir:

- La política debe ser denegar todo de forma predeterminada, a excepción de los recursos públicos.
- Se debe minimizar el uso de intercambio de recursos HTTP de origen cruzado (CORS).
- Se debe deshabilitar el listado de directorios del servidor web.
- Registrar errores de control de acceso que alerten a los administradores.
- Limitar la tasa de acceso a las APIs.
- Los tokens JSON (JWT) deben ser invalidados tras la finalización de la sesión. Los tokens JWT sin estado deben ser de corta duración para minimizar la ventana de oportunidad para un atacante. Para los JWT de vida más larga, considere el uso de tokens de actualización y seguir los estándares OAuth para revocar el acceso.
- Utilizar kits de herramientas o patrones con controles de acceso simples y declarativos.
- Incluir pruebas de control de acceso en las pruebas unitarias y de integración.

3.2.2 A02:2025 Configuración de seguridad incorrecta

Establecer una configuración por defecto o directamente una falta de configuración.

Cómo prevenir:

- Implementar procesos seguros de instalación sin funcionalidades innecesarias.
- Eliminar o no instalar frameworks y funcionalidades no utilizadas.
- Una tarea para revisar y actualizar las configuraciones apropiadas para todas las notas de seguridad, actualizaciones y parches como parte del proceso de gestión de parches. Revisar los permisos de almacenamiento en la nube (por ejemplo, permisos de bucket S3).

- Entornos de desarrollo, preproducción y producción configurados de manera idéntica y con diferentes credenciales para cada entorno.
- Las aplicaciones deben tener arquitecturas segmentadas que proporcionen una separación efectiva y segura entre componentes.
- Enviar directivas de seguridad a los clientes, por ejemplo, encabezados de seguridad.
- Un proceso automatizado para verificar la efectividad de las configuraciones y configuraciones en todos los entornos. Si estas verificaciones no están automatizadas, deben verificarse a mano (mínimo 1 al año).
- Añadir de forma proactiva una configuración central para interceptar mensajes de error excesivos como una copia de seguridad.
- Utilizar la federación de identidades, credenciales de corta duración o los mecanismos de acceso basados en roles (RBAC) o atributos (ABAC).

3.2.3 A03:2025 Uso de componentes con vulnerabilidades conocidas

Los fallos de la cadena de suministro de software son deficiencias en el proceso de construcción, distribución o actualización de software. A menudo son causados por vulnerabilidades o cambios maliciosos en código de terceros, herramientas u otras dependencias en las que se basa el sistema.

Cómo prevenir:

- Gestionar de forma centralizada la lista de materiales de software (SBOM) de todo el software.
- Rastrear no solo sus dependencias directas, sino también sus dependencias (transitivas), etc.
- Reducir la superficie de ataque eliminando dependencias no utilizadas, características innecesarias, componentes, archivos y documentación. Utilizar herramientas para mantener un inventario de versiones de componentes.
- Monitorizar continuamente fuentes como CVE (<https://cve.mitre.org/>) y NVD (<https://nvd.nist.gov/>) en búsqueda de vulnerabilidades en los componentes utilizados.
- Utilizar herramientas de análisis automatizados.
- Suscribirse a alertas de seguridad.
- Obtener componentes únicamente de orígenes oficiales.
- Supervisar bibliotecas y componentes que no poseen mantenimiento.
- Monitorizar, evaluar y aplicar actualizaciones.
- Actualizar su CI/CD, IDE y cualquier otra herramienta para desarrolladores regularmente.
- Evitar implementar actualizaciones en todos los sistemas simultáneamente. Utilizar implementaciones por etapas o implementaciones canarias para limitar la exposición en caso de que un proveedor de confianza se vea comprometido.
- Debe haber un proceso de gestión de cambios o un sistema de seguimiento para realizar un seguimiento de los cambios a:
 - Configuración de CI/CD.
 - Repositorios de código.
 - Zonas de Sandbox.
 - Desarrollador IDEs.
 - Herramientas de SBOM y artefactos creados.
 - Sistemas de registro y registros.
 - Integraciones de terceros, como SaaS.

- Repositorios de artefactos.
- Registros de contenedores.

Cada organización debe garantizar un plan continuo para monitorizar, clasificar y aplicar actualizaciones o cambios de configuración durante la vida útil de la aplicación.

3.2.4 A04:2025 Exposición de datos sensibles

Defensas ante los ataques de exposición de datos sensibles, esto debe aplicarse no sólo al cifrado de los datos en tránsito sino también en reposo, así como una correcta gestión de los mismos por parte de los usuarios. Datos sensibles pueden ser entre otros, las contraseñas, los números de tarjetas de crédito, los registros de salud, la información personal y los secretos comerciales requieren protección adicional, especialmente si esos datos entran en las leyes de privacidad, por ejemplo, el Reglamento General de Protección de Datos (GDPR) o regulaciones como el Estándar de Seguridad de Datos PCI (PCI DSS).

Cómo prevenir:

- Clasificar y etiquetar los datos procesados, almacenados o transmitidos por una aplicación. Identificar qué datos son confidenciales de acuerdo con las leyes de privacidad, los requisitos reglamentarios o las necesidades comerciales.
- No almacenar datos sensibles innecesariamente.
- Utilizar implementaciones confiables de algoritmos criptográficos siempre que sea posible.
- Cifrar todos los datos confidenciales en reposo.
- Utilizar únicamente algoritmos y protocolos estándares y fuertes.
- Cifrar todos los datos en tránsito ($\geq$ TLS 1.2 solamente, con cifrados Forward Secrecy (FS) o soporte de algoritmos de cambio de clave cuántica). Para HTTPS, aplique el cifrado utilizando HTTP Strict Transport Security (HSTS).
- Deshabilitar el almacenamiento en caché para las respuestas que contienen datos confidenciales. Esto incluye el almacenamiento en caché en su CDN, servidor web y cualquier almacenamiento en caché de aplicaciones (por ejemplo: Redis).
- Aplicar los controles de seguridad requeridos según la clasificación de datos.
- Evitar protocolos no cifrados como FTP y STARTTLS. Evite el uso de SMTP para la transmisión de datos confidenciales.
- Almacenar contraseñas utilizando funciones de hashing.
- Los vectores de inicialización deben elegirse apropiados para el modo de funcionamiento.
- Utilizar el cifrado autenticado en lugar de solo el cifrado.
- Las claves deben generarse criptográficamente de forma aleatoria y almacenarse en la memoria como matrices de bytes.
- Evitar funciones criptográficas obsoletas, métodos de construcción de bloques y esquemas de relleno, tales como MD5, SHA1, Cipher Block Chaining Mode (CBC), PKCS número 1 v1.5.
- Prepararse para la criptografía cuántica posterior (PQC, ver referencia de ENISA) para que los sistemas de alto riesgo estén seguros a más tardar a finales de 2030.

3.2.5 A05:2025 Inyección

Se produce al enviar datos no confiables a un intérprete (por ejemplo, un navegador, una base de datos, la línea de comandos). Los datos dañinos del atacante pueden engañar al intérprete para que acceda a información no autorizada o para que se ejecuten comandos involuntarios. Una aplicación

es vulnerable a este tipo de ataques cuando los datos suministrados por el usuario no son validados y filtrados correctamente por la aplicación.

La detección se optimiza mediante una combinación de revisión de código fuente junto con pruebas automatizadas (incluidas las descargas) de todos los parámetros, encabezados, URL, cookies, JSON, SOAP y entradas de datos XML. La adición de herramientas de prueba de seguridad de aplicaciones estáticas (SAST), dinámicas (DAST) e interactivas (IAST) en la canalización de CI/CD también puede ser útil para identificar fallos de inyección antes de la implementación en producción.

Algunas de las inyecciones más comunes son:

- **Inyección SQL:** Insertar caracteres especiales SQL en los campos de entrada de una aplicación Web con el objetivo de acceder a una cuenta o modificar los datos.
- **Inyección SQL ciega (*Blind SQL*):** Obtener información sensible realizando consultas en las que las únicas opciones de respuesta posibles son verdadero o falso.
- **Inyección en base de datos NoSQL:** Debido a que estos ataques de inyección NoSQL pueden ejecutarse dentro de un lenguaje de procedimiento, en lugar de en el lenguaje declarativo de SQL, los impactos potenciales son mayores que la inyección de SQL tradicional.
- **Inyección LDAP (& XPath):** La inyección de LDAP se produce cuando la entrada del usuario no está correctamente validada y se utiliza como parte de un filtro LDAP generado dinámicamente. Esto resulta en una posible manipulación de las declaraciones LDAP realizadas en el servidor LDAP permitiendo ver, modificar u omitir las credenciales de autenticación. Los ataques XPath Injection ocurren cuando un sitio web usa la información que introduce el usuario para construir una consulta (query) xpath para los datos XML, para desconcertar al servidor cuando analiza el sentido de la consulta XPath, provocando la revelación de contenido XML al cual el cliente no debería tener acceso.
- **Inyección EL:** Los motores de Expression Language (EL, comunes en Java EE, JSP, Spring, Struts) permiten incrustar código para acceder a datos, y si un atacante controla esta entrada, puede ejecutar código arbitrario, robar datos sensibles o comprometer el servidor por completo. También se producen en subtipos de EL como el Object Graph Navigation Library (OGNL).

Una clase relacionada de vulnerabilidades de inyección se ha vuelto común en los LLM. Estos se discuten por separado en el OWASP LLM Top 10, específicamente [LLM01:2025 Prompt Injection](#).

Cómo prevenir:

- Separar los datos de los comandos y las consultas.
- Minimizar los permisos de la aplicación Web en la Base de Datos.
- Filtrar palabras SQL reservadas (y meta caracteres).
- Utilizar consultas parametrizadas y procedimientos almacenados.
- Realizar validaciones de entradas de datos en el servidor utilizando "listas blancas".
- Limitar mensajes de error.
- Utilizar firewalls de aplicación (WAF).
- Utilizar LIMIT y otros controles SQL.

3.2.6 A06:2025 Diseño inseguro

El diseño inseguro es una categoría amplia, creada en 2021, que representa diferentes debilidades, expresadas como "diseño de control ausente o ineficaz". Hay una diferencia entre diseño inseguro e

implementación insegura. Ambos conceptos tienen diferentes causas de origen y remediación. Un diseño seguro puede seguir teniendo defectos de implementación que den lugar a vulnerabilidades que puedan ser explotadas. Un diseño inseguro no puede arreglarse con una implementación perfecta, ya que, por definición, los controles de seguridad necesarios nunca se crearon para defenderse de ataques específicos. Uno de los factores que contribuyen a un diseño inseguro es la falta de un perfil de riesgo empresarial inherente al software o al sistema que se está desarrollando y, por tanto, la incapacidad de determinar qué nivel de diseño de seguridad se requiere.

Cómo prevenir:

- Establecer y utilizar un ciclo de vida de desarrollo seguro con los responsables de seguridad para ayudar a evaluar y diseñar controles relacionados con la seguridad y la privacidad.
- Establecer y utilizar una biblioteca de patrones de diseño seguros.
- Utilizar el modelado de amenazas para partes críticas de la aplicación, como autenticación, control de acceso, lógica de negocio y flujos de claves.
- Formación integral a los usuarios sobre el uso seguro de las aplicaciones y sistemas.
- Integrar el lenguaje de seguridad y los controles en las historias de los usuarios y en cada nivel de la aplicación (desde frontend hasta backend).
- Escribir pruebas unitarias e integración para validar que todos los flujos críticos son resistentes al modelo de amenaza. Compilar casos de uso para cada nivel de la aplicación.
- Segregar las capas de nivel en el sistema y las capas de red, dependiendo de las necesidades de exposición y protección.
- Segregar a los clientes robustamente por diseño en todos los niveles.

3.2.7 A07:2025 Pérdida de autenticación

Cuando un atacante es capaz de engañar a un sistema para que reconozca a un usuario inválido o incorrecto como legítimo, esta vulnerabilidad está presente.

Cómo prevenir:

- Implementar y aplicar el uso de la autenticación multifactor (MFA), para evitar el relleno de credenciales automatizados, la fuerza bruta y los ataques de reutilización de credenciales robados.
- Fomentar y habilitar el uso de administradores de contraseñas.
- No enviar ni implementar con credenciales predeterminadas, débiles o conocidas.
- Implementar la política de seguridad para establecer duración de las contraseñas, longitud, complejidad, etc.
- Hay que asegurar que el registro, la recuperación de credenciales y las vías de la API se protejan contra los ataques de enumeración de cuentas mediante el uso de los mismos mensajes para todos los resultados (“Nombre de usuario o contraseña no válida”).
- Limitar o incrementar el tiempo de respuesta de cada intento fallido de inicio de sesión.
- Registrar todos los fallos de inicio de sesión.
- Utilizar un gestor de sesiones que genere un nuevo ID de sesión aleatorio.
- El Session-ID no debe incluirse en la URL.

3.2.8 A08:2025 Software or Data Integrity Failures

Los fallos de integridad de software y datos se refieren a código o infraestructura que no protege contra el código o datos inválidos o no confiables y se acaban tratando válidos y de confianza. El concepto de la lista de 2017 de deserialización insegura entraría en este grupo.

Cómo prevenir.

- Utilizar firmas digitales o mecanismos similares para verificar que el software o los datos provienen de la fuente esperada y no se ha modificado.
- Asegurarse de que las bibliotecas y dependencias, como npm o Maven, solo consumen repositorios de confianza.
- Garantizar que hay un proceso de revisión de los cambios de código y configuración.
- Asegurarse de la protección de la integridad del código en todo el pipeline de CI/CD.
- Asegurarse de que los datos serializados no firmados o no cifrados no se reciban de clientes no confiables.
- Exigir el cumplimiento estricto de verificaciones de tipo de dato.
- Aislar el código que realiza la deserialización.
- Registrar las excepciones y fallas en la deserialización.
- Monitorizar los procesos de deserialización.
- Restringir y monitorizar la conexión de red desde contenedores o servidores que utilizan funcionalidades de deserialización.

3.2.9 A09:2025 Registro, monitorización y alerta insuficientes

Sin registro y monitorización, los ataques y las brechas no se pueden detectar, y sin alertarlo es muy difícil responder de manera rápida y efectiva durante un incidente de seguridad.

Cómo prevenir:

- Asegúrese de que todos los fallos de inicio de sesión, control de acceso y validación de entrada del lado del servidor se puedan registrar con suficiente contexto de usuario para identificar cuentas sospechosas o maliciosas y tener suficiente tiempo para permitir un análisis forense retrasado.
- Asegúrese de que cada parte de su aplicación que contiene un control de seguridad esté registrada, ya sea que tenga éxito o falle.
- Asegúrese de que los registros se generan en un formato que las soluciones de gestión de registros pueden consumir fácilmente.
- Asegúrese de que los datos de registro estén codificados correctamente para evitar inyecciones o ataques en los sistemas de registro o monitorización.
- Asegúrese de que todas las transacciones tengan una pista de auditoría con controles de integridad para evitar la manipulación o eliminación.
- Asegúrese de que todas las transacciones que arrojan un error se revierten y se inician de nuevo.
- DevSecOps y los equipos de seguridad deben establecer casos de uso de monitorización y alerta efectivos, de modo que las actividades sospechosas sean detectadas y respondidas rápidamente por el equipo del Centro de Operaciones de Seguridad (SOC, en inglés).
- Agregue 'honeytokens' como trampas para atacantes en su aplicación, por ejemplo, en la base de datos, datos, como identidad de usuario real y/o técnica.

- El análisis del comportamiento y el soporte de IA podrían ser opcionalmente una técnica adicional para soportar bajas tasas de falsos positivos para las alertas.
- Establecer o adoptar un plan de respuesta o recuperación de incidentes.

Hay productos de protección de aplicaciones comerciales y de código abierto, como el conjunto de reglas básicas de ModSecurity de OWASP y el software de correlación de registro de código abierto, como la pila Elasticsearch, Logstash, Kibana (ELK), que cuentan con paneles personalizados y alertas que pueden ayudar a combatir estos problemas. También hay herramientas de observabilidad comercial que pueden ayudar a responder o bloquear ataques en tiempo cercano al real.

Igualmente existen herramientas para la gestión de tickets de incidentes de seguridad en el sector público como los ofrecidos por el Centro Criptológico Nacional mediante la herramienta CCN LUCIA.

3.2.10 A10:2025 Manejo incorrecto de condiciones excepcionales

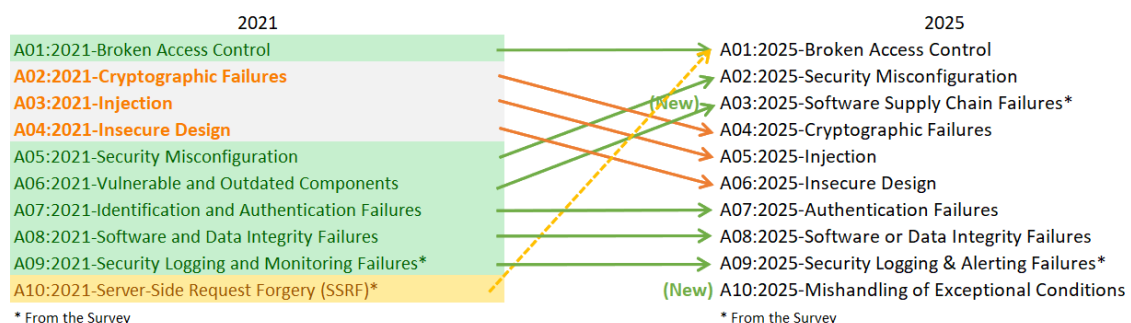
El mal manejo de las condiciones excepcionales en el software ocurre cuando los programas no previenen, detectan y responden a situaciones inusuales e impredecibles, lo que conduce a bloqueos, comportamientos inesperados y, a veces, vulnerabilidades. Esto puede implicar una o más de los siguientes tres fallos: la aplicación no impide que ocurra una situación inusual, no identifica la situación a medida que sucede y/o responde mal o no responde en absoluto a la situación después. Esta categoría busca asegurar que cuando un sistema falla, lo haga de manera controlada y segura, sin exponer la integridad ni la disponibilidad del sistema.

Cómo prevenir.

- Implementar un manejo de excepciones robusto: usar un único manejador global de excepciones para asegurar que todos los errores se procesen de la misma forma.
- Validación de datos estricta: Implementar validación de entradas robusta para prevenir que datos malformados lleguen a capas lógicas vulnerables.
- Registros seguros sin información confidencial.
- Mensajes genéricos: mostrar errores amigables al usuario final y registrar los detalles técnicos solo en logs internos seguros.
- Diseño a prueba de fallos: configurar los sistemas para que, ante cualquier fallo crítico, la acción por defecto sea prohibir el acceso o la operación.
- Efectuar pruebas de estrés de los sistemas de información.

3.3 OWASP cuadro resumen

Los diez riesgos más críticos en Aplicaciones Web según OWASP Top 10 - 2025 son:



Hay dos nuevas categorías y una consolidación en el Top Ten para 2025:

- A01:2025 – Pérdida de control de acceso mantiene su posición en el #1 como el riesgo de seguridad de la aplicación más grave. Como se indica por la línea discontinua en la figura anterior, la falsificación de solicitud del lado del servidor (SSRF) se ha introducido en esta categoría.
- A02:2025 - La configuración de seguridad incorrecta de seguridad subió del #5 en 2021 al #2 en 2025. Las configuraciones erróneas son más frecuentes en los datos de este ciclo. El 3,00% de las aplicaciones probadas tenían uno o más de los 16 CWE de esta categoría. Esto no es sorprendente, ya que la ingeniería de software continúa aumentando la cantidad de comportamiento de una aplicación que se basa en configuraciones.
- A03:2025 – Uso de componentes con vulnerabilidades conocidas es una expansión de A06:2021- Componentes vulnerables y obsoletos para incluir un alcance más amplio de compromisos que ocurren dentro o a través de todo el ecosistema de dependencias de software, sistemas de construcción e infraestructura de distribución.
- A04:2025 - Los fallos criptográficos (exposición de datos sensible) caen dos lugares del #2 al #4 en el ranking. El punto clave es la criptografía utilizada o la falta de esta, tanto en datos almacenados como en tránsito.
- A05:2025 - La inyección cae dos puntos del #3 al #5 en el ranking, manteniendo su posición en relación con las fallas criptográficas y el diseño inseguro. La inyección incluye una variedad de problemas, desde secuencias de comandos entre sitios (alta frecuencia/bajo impacto) hasta vulnerabilidades de inyección SQL (baja frecuencia/alto impacto).
- A06:2025 – Diseño inseguro desliza dos lugares del #4 al #6 en el ranking como Configuración errónea de seguridad y fallas de la cadena de suministro de software saltaron. Esta categoría se introdujo en 2021.
- A07:2025 - Los fallos de autenticación. Esta categoría sigue siendo importante, pero el mayor uso de marcos estandarizados para la autenticación parece estar teniendo efectos beneficiosos sobre los casos de fallos de autenticación.
- A08:2025 - Los fallos de software o de integridad de datos continúan en el #8 de la lista. Esta categoría se centra en la falta de mantenimiento de los límites de confianza y verificar la integridad del software, el código y los artefactos de datos a un nivel más bajo que los fallos de la cadena de suministro de software.
- A09:2025 - El registro de seguridad y las fallas de alertas mantienen su posición en el #9. Esta categoría tiene un ligero cambio de nombre para enfatizar la importancia de la funcionalidad de alerta necesaria para inducir la acción apropiada en eventos de registro relevantes.
- A10:2025 – Manejo incorrecto de condiciones excepcionales es una nueva categoría para 2025. Esta categoría se centra en el manejo incorrecto de errores, errores lógicos, fallas abiertas y otros escenarios relacionados, derivados de condiciones anormales que los sistemas pueden encontrar.

3.3.1 Otros ataques

- **Desplazamientos por directorios (*path traversal*):** permiten a un usuario acceder sin control a ficheros y directorios que se almacenan fuera de la carpeta raíz de la aplicación Web.
- **Cross-Site Tracing (XST):** Es un tipo de *Cross Site Scripting* (XSS) que utiliza el método HTTP TRACE que se utiliza principalmente para fines de depuración repitiendo toda la información enviada por el cliente al servidor (petición “echo”). Este método viene activado por defecto en muchos servidores.

- **Cross-Site Request Forgery (CSRF):** fuerza al navegador web de una víctima que está logado en algún servicio legítimo a enviar una petición a una aplicación web vulnerable con el fin de que la petición se procese en el nombre de la víctima.
- **Remote file inclusion (RFI) y Local File Inclusion (LFI):** RFI permite a un atacante ejecutar archivos remotos alojados en otros servidores. Esta vulnerabilidad existe solamente en páginas dinámicas en PHP. LFI es similar a RFI excepto en que en lugar de incluir ficheros remotos sólo se pueden incluir para ejecución ficheros alojados en el mismo servidor víctima.
- **Ataques contra SSL/TLS:** ver temas de seguridad en redes.
- **Fallos en el manejo de memoria:** todavía hay muchos sistemas heredados en la producción en todo el mundo, nuevos sistemas de bajo nivel que requieren el uso de lenguajes seguros que no son de memoria, y aplicaciones web que interactúan con mainframes, dispositivos IoT, firmware y otros sistemas que pueden verse obligados a administrar su propia memoria.
- **Confianza inapropiada en código generado por IA:** estamos viendo que las prácticas de desarrollo de software cambian para incluir no solo el código escrito con la ayuda de la IA, sino también el código escrito y puesto en producción casi por completo sin la supervisión humana (Vibe coding).

3.4 Metodologías de seguridad en aplicaciones web

Una metodología de seguridad en aplicaciones web debe tener en cuenta lo siguiente:

- La seguridad en el ciclo de vida de desarrollo.
- Realizar pruebas de caja negra.
- Realizar pruebas de caja blanca.
- Implementar firewalls de nivel de aplicación (Web Application Firewalls, WAF).

OWASP ha desarrollado un marco de trabajo abierto denominado **SAMM** (*Software Assurance Maturity Model*) que pretende ayudar a las organizaciones en el proceso de formulación y aplicación de estrategias de seguridad en el software.

El CCN-CERT ha elaborado una [guía de seguridad para los entornos y las aplicaciones Web: CCN-STIC-812](#). En cuanto a servidores web también se ha publicado la guía [CCN-STIC-673](#).

3.5 La seguridad desde el punto de vista de la arquitectura de sistemas

Si bien las tecnologías de seguridad pueden aplicarse en las diferentes arquitecturas de sistemas, como arquitecturas monolíticas o basadas en microservicios, sí que existen aspectos diferenciadores a tener en cuenta a la hora de planificar y gestionar la seguridad. A continuación, se establecen algunas características de cada modelo:

Seguridad en Arquitectura Monolítica:

- **Centralizada:** la seguridad se gestiona en un solo lugar (autenticación, autorización, validación de datos).
- **Menor superficie de ataque:** al no haber servicios expuestos al exterior, la comunicación interna es más segura.
- **Simple:** más fácil de configurar, auditar y controlar al ser una sola unidad.
- **Riesgo:** si el perímetro de seguridad es comprometido, el atacante obtiene acceso a toda la aplicación.

Seguridad en Microservicios

- **Granular (defensa en profundidad):** cada microservicio puede tener sus propias políticas de seguridad y base de datos.
- **Alta superficie de ataque:** cada servicio expuesto aumenta los puntos de vulnerabilidad.
- **Complejidad:** requiere gestionar autenticación y autorización en múltiples componentes (API Gateways, servicios, redes).
- **Resiliencia:** si un servicio falla, la seguridad de los demás mantiene la integridad del sistema.
- **Recomendación:** uso obligado de protocolos de seguridad modernos (OAuth2, JWT) y, preferiblemente, un enfoque de **Zero Trust**.

4 Seguridad en servicios web

En sus inicios, las APIs eran utilizadas principalmente para la integración de *software* local, facilitando la comunicación entre programas dentro de un mismo sistema operativo. Con la llegada de la web y el modelo cliente-servidor, surgieron las APIs web, que permitieron la interacción entre aplicaciones remotas a través de protocolos como SOAP y REST. Esta evolución transformó la forma en que se diseñaban las arquitecturas de *software*, permitiendo que distintas plataformas se conectaran y compartieran información sin necesidad de estar alojadas en el mismo entorno.

El gran impulso de las APIs llegó con la expansión de los servicios en la nube y la popularización de los dispositivos móviles. Empresas como Google, Amazon, Facebook y Microsoft comenzaron a ofrecer APIs públicas que permitían a los desarrolladores integrar funcionalidades como autenticación, mapas, pagos y almacenamiento sin necesidad de desarrollar esas capacidades desde cero. Este modelo permitió la escalabilidad y la interoperabilidad entre servicios, reduciendo costos y tiempos de desarrollo.

Sin embargo, este crecimiento también ha traído consigo **nuevos desafíos de seguridad**. La exposición de datos sensibles, las malas configuraciones y la falta de controles de acceso han convertido a las APIs en un objetivo prioritario para atacantes.

Principales riesgos en seguridad de API

Las APIs desempeñan un papel crucial como puente entre aplicaciones, pero su **exposición pública** las convierte en un objetivo frecuente para atacantes. A continuación, exploramos los principales riesgos de seguridad en APIs basados en el [OWASP API SecurityTop 10](#) (2023):

1. **Autorización a nivel de objeto rota (BOLA - Broken Object Level Authorization):** muchas APIs permiten a los usuarios recuperar datos basados en un identificador (ID) sin verificar si tienen permiso para acceder a ellos, lo que facilita el acceso no autorizado a información sensible.
2. **Autenticación rota:** implementaciones débiles de autenticación pueden permitir que atacantes se hagan pasar por otros usuarios o accedan a datos sin restricciones. Un caso típico ocurre cuando una API permite **tokens de sesión sin caducidad**.
3. **Autorización a nivel de propiedad de objeto rota:** ocurre cuando una API permite a un usuario acceder o modificar propiedades de un objeto sin validar correctamente los permisos.
4. **Consumo de recursos sin restricciones:** las APIs requieren recursos como ancho de banda, CPU y memoria, y sin límites adecuados, pueden ser explotadas para causar Denegación de Servicio (DoS) o aumentar costos operativos.

5. **Autorización a nivel de función rota:** las diferencias entre usuarios regulares y administradores deben estar bien definidas, pero algunas APIs no aplican controles adecuados.
6. **Acceso sin restricciones a flujos de negocio sensibles:** las APIs pueden exponer funciones críticas sin prever su posible abuso automatizado.
7. **Server-Side Request Forgery (SSRF):** Si una API permite a los usuarios proporcionar una URL externa para extraer datos, pero no valida el destino, un atacante podría utilizarla para enviar solicitudes maliciosas a servidores internos.
8. **Configuración de seguridad incorrecta:** muchas APIs vienen con configuraciones por defecto que exponen información sensible o permiten accesos inseguros.
9. **Gestión inadecuada del inventario:** las APIs deben mantener un inventario actualizado de versiones y *endpoints* para evitar exponer funciones obsoletas o vulnerables.
10. **Consumo inseguro de APIs:** los desarrolladores suelen confiar en datos de APIs de terceros sin validación adecuada, lo que permite ataques indirectos.

Tanto si se implementa la API con servicios SOAP o servicios REST, existen medidas que se pueden tomar para fortalecer la seguridad:

- Utilizar tokens. Configurar identidades confiables y controle el acceso a los servicios y a los recursos utilizando los tokens asignados a dichas identidades. Usar tecnologías de autenticación y control de acceso como JWT enmarcado en el framework OAuth 2.0 y API Keys, así como la expiración de tokens para limitar su tiempo de uso y evitar accesos prolongados en caso de compromiso.
- Utilizar métodos de cifrado y firmas. Cifrar los datos mediante un método como TLS. Solicitar el uso de firmas para asegurar que solamente los usuarios adecuados descifren y modifiquen sus datos.
- Identificar las vulnerabilidades, por ejemplo, utilizando un escáner web de vulnerabilidades como OWASP ZAP. Mantenga actualizados sus elementos de API, sus controladores, redes y su sistema operativo. Manténgase al tanto de cómo funciona todo en conjunto, e identifique las debilidades que se podrían utilizar para entrar en sus API. Utilice analizadores de protocolos para detectar los problemas de seguridad y rastrear las pérdidas de datos.
- Utilizar cupos y límites. Establecer un cupo en la frecuencia con la que se puede recurrir al API, y dar seguimiento al historial de uso. Encontrar más solicitudes a una API puede indicar un abuso. También podría ser un error de programación, como una solicitud a la API en un bucle sin fin. Establecer reglas de limitación para proteger las APIs de ataques de denegación de servicio y picos de uso.
- Utilizar una puerta de enlace de API. Las puertas de enlace de API funcionan como el principal punto de control para el tráfico de las API. Una buena puerta de enlace le permitirá autenticar el tráfico, así como controlar y analizar cómo se utilizan sus API.
- La gestión de datos es clave para evitar la exposición innecesaria de información en las APIs. Limitar los datos que una API devuelve es fundamental para reducir el riesgo de filtraciones, y esto se puede lograr mediante *field filtering* (selección de campos), donde los clientes solo reciben la información estrictamente necesaria.

Existen numerosos tipos de APIs en la industria actualmente como SOAP, REST, GraphQL, gRPC y MQTT entre otros. En los siguientes apartados describiremos los dos primeros ya que tienen una mayor implantación, sin embargo, cada uno puede satisfacer un propósito diferente y se debe estudiar las características de cada uno de cara de decidir cuales implementar para cumplir con los requisitos de negocio. Como refresco, se presenta una tabla con las características diferenciadas más destacadas entre las API SOAP y REST.

	SOAP	REST
Significado	Protocolo simple de acceso a objetos	Transferencia de estado representacional
¿Qué es?	SOAP es un protocolo para la comunicación entre aplicaciones	REST es un estilo de arquitectura para diseñar interfaces de comunicación.
Diseño	La API de SOAP expone la operación.	La API de REST expone los datos.
Protocolo de transporte	SOAP es independiente y puede funcionar con cualquier protocolo de transporte.	REST solo funciona con HTTPS.
Formato de datos	SOAP solo admite el intercambio de datos XML.	REST admite XML, JSON, texto plano y HTML.
Rendimiento	Los mensajes SOAP son más grandes, lo que hace que la comunicación sea más lenta.	REST tiene un rendimiento más rápido debido a los mensajes más pequeños y al soporte de almacenamiento en caché.
Escalabilidad	SOAP es difícil de escalar. El servidor mantiene el estado al almacenar todos los mensajes anteriores intercambiados con un cliente.	REST es fácil de escalar. No tiene estado, por lo que cada mensaje se procesa de manera independiente de los mensajes anteriores.
Seguridad	SOAP admite el cifrado con sobrecargas adicionales.	REST admite cifrado sin afectar al rendimiento.

4.1 API REST

La arquitectura de API de transferencia de estado representacional, o API REST, utiliza la transferencia de datos JSON y el protocolo de transferencia HTTP/S; en comparación con otras arquitecturas de API, simplifican el desarrollo de las API REST. Las API RESTful emplean solicitudes HTTP para crear (POST), actualizar (PUT), leer (GET) y eliminar (DELETE) datos.

La seguridad de las API REST, que carecen de unas disposiciones de seguridad propias, depende del diseño de la API. Los servicios de transmisión de datos, implementación e interacción con los clientes deben incorporar ciertas consideraciones de seguridad. La mayoría de las API RESTful usarán métodos de seguridad de la capa de transporte (como HTTPS) y autenticación basada en identificadores.

Una opción arquitectónica habitual para proteger la seguridad de las API REST consiste en implementarlas detrás de una puerta de enlace de API y, a continuación, ofrecer esta opción de conectividad a los clientes. No obstante, hay que tener en cuenta que una puerta de enlace de API no está equipada para defender de todas las amenazas de seguridad.

4.2 API SOAP

SOAP (protocolo simple de acceso a objetos) es un protocolo para intercambiar datos estructurados en la implementación de servicios web en redes de ordenadores. SOAP utiliza XML como formato de mensajes y puede transmitirse a través de distintos protocolos de niveles inferiores, como HTTP y SMTP. Las API SOAP suelen protegerse mediante una combinación de seguridad de la capa de transporte (como HTTPS) y en el nivel del mensaje (como XML, las firmas digitales y el cifrado).

La seguridad de las API SOAP afecta a las extensiones de los protocolos. SOAP se adhiere a las especificaciones de los servicios web (WS) y a la extensión Web Services Security que veremos más en detalle a continuación.

4.3 WS-Security

WS-Security (WSS) es una extensión al estándar SOAP cuyo objetivo es proporcionar mecanismos de seguridad a los mensajes SOAP. En concreto, proporcionar autenticación, integridad y confidencialidad en el intercambio de estos. Al contrario que otras normas, WSS no desarrolla estándares de nuevas tecnologías para cifrado, firma o autenticación, sino que se usa para que las partes indiquen qué mecanismos concretos de seguridad están empleando mediante extensiones XML en la cabecera de los mensajes SOAP.

Es decir, WS-Security no desarrolla una nueva tecnología de seguridad, sino que especifica cómo usar una serie de tecnologías y estándares de seguridad comúnmente aceptados para autenticación (por ejemplo, aserciones SAML o un certificado X.509 con una prueba de identidad), cifrado del mensaje (XML Encryption) y firma (XML DSignature).

De esta forma un proveedor de servicios web puede recibir un mensaje SOAP y mediante la lectura de las extensiones WSS de cabecera del mensaje SOAP identificar qué mecanismos ha escogido el emisor de este para autenticar al usuario final, cifrar el mensaje y firmarlo y de esta forma proceder a su validación. El objetivo final es ofrecer mecanismos de interoperabilidad que permitan la seguridad en los servicios web.

La especificación WS-Security fue originalmente propuesta por Microsoft, IBM y VeriSign en abril de 2002, y aprobada y estandarizada por OASIS en abril de 2004. Hoy en día WS-Security se está convirtiendo en el método principal para asegurar Servicios Web.

4.3.1 Seguridad a nivel de transporte (TLS) y a nivel de mensaje (WSS)

Es importante considerar la diferencia entre WSS y TLS (Transport Layer Security). TLS ofrece un canal seguro entre dos extremos de nivel de transporte; normalmente el navegador del usuario y el servidor web si se emplea con HTTP (HTTPS). Por su parte, WSS está enfocado a garantizar la seguridad del mensaje SOAP de nivel de aplicación por sí mismo, independiente del canal de transporte que podría ser TCP o no. Además, el mensaje SOAP podría también mediante extensiones WSS incluir varias credenciales de autenticación que la aplicación proveedora identificaría e interpretaría por separado de acuerdo con su propia lógica. Las componentes de seguridad del mensaje SOAP permanecen independientemente del destino final del mensaje y del número de intermediarios. TLS, en cambio ofrece mecanismos seguros para el nivel de transporte, pero su alcance finaliza en el punto de terminación de la conexión a nivel de transporte TCP. Por ello, ambos protocolos son compatibles. Así, si se utiliza SOAP sobre HTTP se podrían asegurar la integridad y confidencialidad de los mensajes SOAP mediante TLS, pero sólo durante una conexión TCP; en los casos de web services multi-hops que impliquen la transmisión de la identidad de usuario final o múltiples identidades, o que requieran asegurar la integridad del mensaje en toda la cadena, no bastaría con TLS.

A continuación, se resumen las diferencias entre los dos mecanismos:

TLS	WSS
Extremo a extremo de nivel de transporte (IP+puerto)	Extremo a extremo de nivel de aplicación.
Credenciales de autenticación de navegador y servidor únicos sin intermediarios.	Credenciales de autenticación de usuario final. Permite múltiples credenciales de autenticación.
No es flexible: se aplica a todo el payload de igual manera durante toda la sesión	Muy flexible: puede aplicarse sólo a una parte del mensaje y discriminar entre Requests y Responses
Se sitúa entre el nivel TCP y el de aplicación	La misma seguridad se puede aplicar en diferentes tecnologías de transporte.

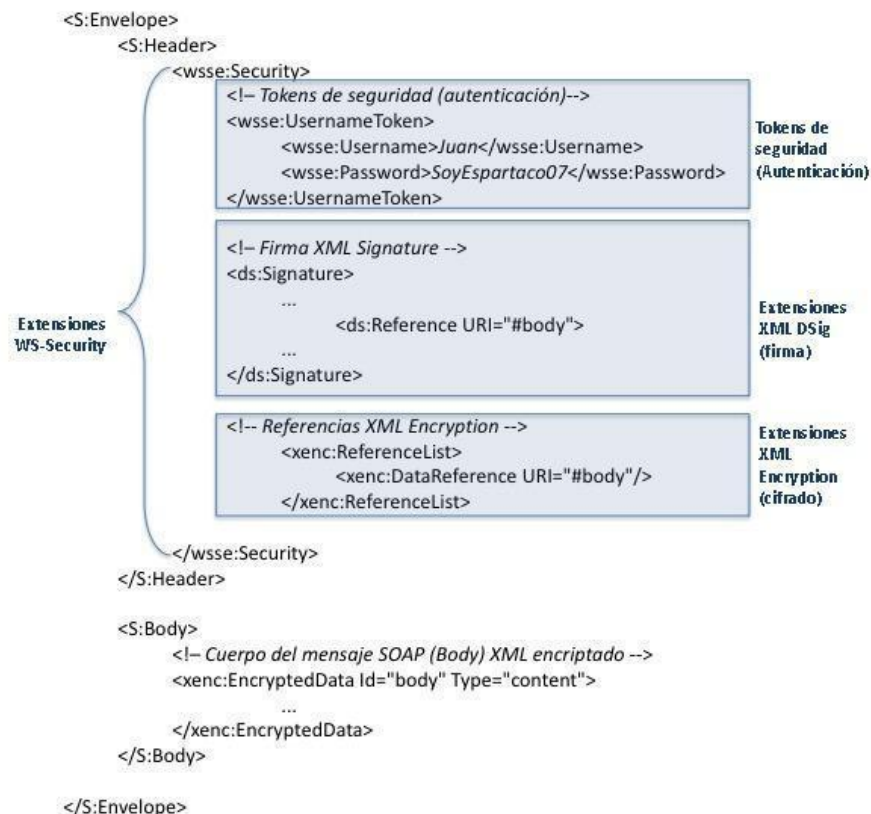
4.3.2 Tecnologías de autenticación, confidencialidad e integridad soportadas por WSS

Como se ha comentado, la norma WSS define extensiones en la cabecera SOAP para la autenticación, integridad y confidencialidad del mensaje. Estas extensiones, al final, están ligadas a las tecnologías de seguridad que se emplean. A continuación, se definen las tecnologías soportadas:

- **Integridad:** WSS emplea **XML Signature (W3C)** para la firma de los mensajes SOAP. Permite firmar cualquier documento XML incluido, naturalmente, mensajes SOAP en su totalidad o sólo partes de él. Para más información consúltase el tema sobre tecnologías de firma electrónica. Un mensaje SOAP puede llevar ninguna, una o múltiples firmas XML Signature.
- **Confidencialidad:** se basa en la especificación **XML-Encryption (W3C)** que al igual que ocurre con XML Signature permite encriptar parte o la totalidad del mensaje.
- **Autenticación:** WS-Security utiliza extensiones XML en la cabecera denominadas *tokens de seguridad*. El estándar WS-Security es muy flexible ya que define extensiones para muchos tipos de tokens posibles y también permite extender el modelo. Algunos ejemplos de tokens disponibles son **username/password**, certificados **X.509**, tickets **Kerberos**, aserciones **SAML**, tokens XrML, tokens XCBF o referencias a URIs. El uso de SAML permite también intercambiar aserciones de autorización para usuarios del servicio.

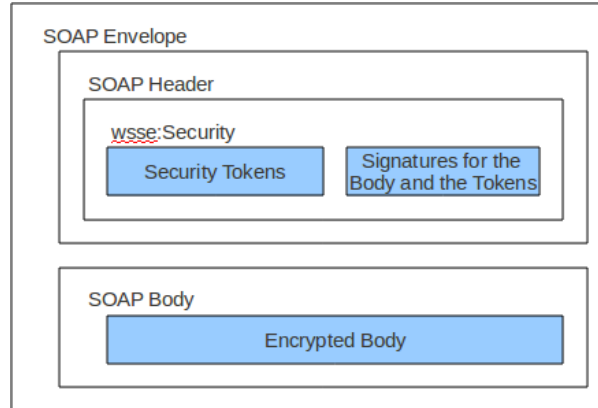
4.3.3 Ejemplo de extensiones WSS

Para su comprensión gráfica se muestra a continuación un esquema de mensaje SOAP con extensión WS-Security. En este caso particular sólo habría una credencial de autenticación que sería el nombre/clave del usuario final, pero, como se ha comentado, podrían incluirse varios tokens de distintas tecnologías (aserciones SAML, Kerberos, certificados X.509 o incluso definidos por el propio usuario) o ninguno, de la misma forma que en el ejemplo sólo se muestra una firma, pero se podrían incluir varias o ninguna. Asimismo, se ha incluido un sólo elemento *ReferenceList* con referencias a segmentos del mensaje que estén cifrados pero se podrían incluir varios o ninguno.



Ejemplo de mensaje SOAP con extensiones WSS

En cuanto a la implementación, la mayoría de los frameworks de desarrollo incluyen APIs para el desarrollo de servicios web con soporte a WSS. En concreto Java ofrece el API Java API for XML Web Services (JAX-WS) y en el entorno .Net el API para construir y consumir Web Services está integrado en Windows Communication Foundation.



Componentes de WS-Security y el mensaje SOAP

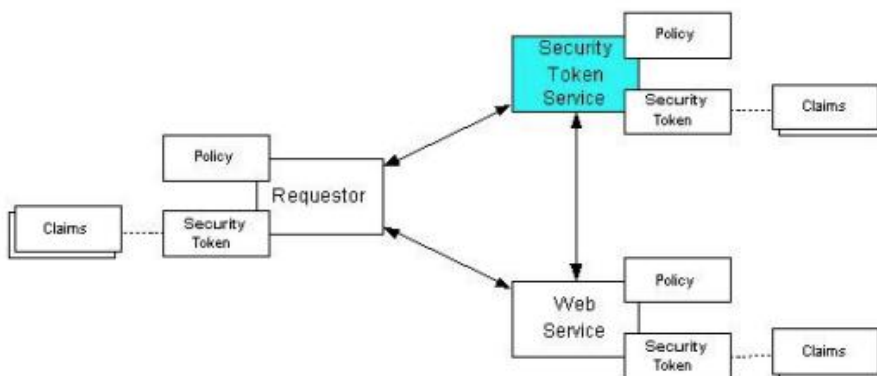
4.3.4 Otras extensiones para web services

WS-Security se usa únicamente para autenticación, confidencialidad (cifrado) e integridad (firma). Existen otras extensiones SOAP **para el ámbito de la seguridad**. Su alcance es distinto del que ofrece WS-Security y por lo tanto todas son compatibles con este estándar. En concreto:

- **WS-Policy** es una serie de extensiones XML que permiten a un servicio web especificar sus requisitos de calidad de servicio y de seguridad para que puedan ser consultadas antes de invocar la interfaz WS. Las extensiones para especificar la tecnología aceptada de seguridad se denominan **WS-SecurityPolicy** y permiten definir qué tokens de autenticación se admiten, qué partes deben estar encriptadas, qué partes firmadas etc.
- **WS-Addressing**. Extensiones XML para la cabecera de los mensajes SOAP que proporciona mecanismos neutrales, independientes del protocolo de transporte, para direccionar mensajes y servicios web. WS-Addressing convierte a los mensajes SOAP en unidades autónomas de comunicación mediante la inclusión en la cabecera de los mensajes información sobre:
 - De dónde viene
 - La dirección de destino (Web Service destino)
 - Destinatario final del servicio: sistema o usuario específico dentro del servicio web de esa dirección de destino a quien va dirigido.
 - Dónde debería ir si no puede ser remitido como estaba previsto
- **WS-Trust**. La especificación WS-Trust define cómo intercambiar tokens de seguridad con credenciales de acceso a servicios web de distintos dominios de confianza. Define un modelo de servicio denominado Security Token Service (STS) y un protocolo para solicitar y entregar tokens de seguridad soportados por WS-Security y que los proveedores de servicios web especifican mediante WS-SecurityPolicy.

El modelo considera 3 actores: el consumidor del servicio web (Requestor), el proveedor del servicio web (Web Service) y un tercero de confianza denominado Servicio de Tokens de Seguridad (Security Tokens Service). De acuerdo con el modelo, si el Requestor no posee credenciales válidas para acceder a un servicio web concreto (no cumple con la política de seguridad) puede solicitar al STS que le haga entrega de las credenciales necesarias (tokens

de seguridad). Si el Receptor cumple la propia política de seguridad del STS, éste le entregará los tokens de seguridad necesarios para el acceso al web service demandado. La siguiente figura muestra un esquema del modelo.



- WS-Federation.** Una Federación es un conjunto de dominios de seguridad que han acordado compartir recursos de forma segura. Está soportado por las extensiones ofrecidas por WS-Security (seguridad en el mensaje SOAP), WS-Trust (intercambio de tokens de seguridad entre distintos dominios de seguridad), and WS-SecurityPolicy (publicidad de la política de seguridad). Las federaciones están compuestas por Proveedores de recursos (Resource Providers), cada uno con su dominio de seguridad. En una Federación, un proveedor de recursos autoriza el acceso a un recurso a un usuario que ha sido avalado (autenticado y autorizado) por otro dominio. Como se ha comentado, se basa en WS-Trust pero su alcance es mayor pues está enfocado a **compartir recursos y a gestionar autorizaciones**. En concreto gestiona la identificación, el registro, la autenticación y la autorización de los usuarios; categoriza los niveles de confianza de las identidades, permite autenticarlas y asignarles atributos para su uso en los servicios web de todos los colaboradores federados.



Pila de protocolos sobre la que se sustentan las extensiones Web Service

- WS-SecureConversation:** Extensiones a WS-Security para el establecimiento de contextos de seguridad para el intercambio de una serie de mensajes SOAP durante toda la sesión que se establezca entre partes de un Web-Service (lo que se denomina una "conversación"). Se diferencia de WS-Security en que ésta norma protege cada uno de los mensajes SOAP de forma independiente, lo que introduce un elevado overhead, mientras que WS-SecureConversation cubre toda la conversación. Con ésto se pretende reducir el overhead

de las comunicaciones al establecer un contexto de seguridad acordado por las partes y que aplica a todos los mensajes intercambiados en la conversación de forma análoga a como se establecen las sesiones en TLS, sólo que en esta ocasión en nivel de los mensajes SOAP.

4.3.5 Crítica a WSS y alternativas

Tanto la arquitectura SOA basada en Web Services SOAP como la propia serie de normas WSS se encuentran muy extendidas. En la Administración General del Estado encontramos ambos estándares en sistemas como la Plataforma de Intermediación de Datos. Se trata de tecnologías maduras que cubren una gran cantidad de escenarios de uso, independientes del contexto introducido por las redes que la sustentan y que cuentan con numerosos APIs y Frameworks de desarrollo ofrecidos por la industria. Son especialmente adecuadas para garantizar la trazabilidad y la interoperabilidad.

La principal crítica realizada a WS sobre SOAP y sus extensiones WSS es el overhead que introducen y su lentitud. Por ello en los últimos años se han popularizado las denominadas arquitecturas RESTful. RESTful es en realidad un concepto de arquitectura de servicios, pero comúnmente - aunque no sea muy acertado - se entiende este término como una alternativa a los servicios web basados en SOAP cuya principal característica es que ofrecen interfaces directas HTTP. Los mensajes van directamente encapsulados en el payload de este protocolo frente a los Web Services SOAP que, aunque en última instancia también se transmiten normalmente sobre HTTP, introducen la capa SOAP y además emplean extensiones XML para seguridad y señalización que los hacen aún más pesados.

Los servicios RESTful son más ligeros y su despliegue suele ser más rápido. No ofrecen soluciones globales interoperables, como pretenden WSS y sus extensiones, pero comúnmente, para los escenarios de servicios concretos que se presentan más a menudo, ofrecen una solución más ligera y rápida en su desarrollo y en sus prestaciones. Su seguridad suele estar garantizada mediante TLS.

Para más información sobre las distintas arquitecturas orientadas a servicios se recomienda estudiar el tema correspondiente.

4.4 SAML

SAML (*Security Assertion Markup Language*) es un lenguaje de marcado desarrollado por el consorcio OASIS para el intercambio de información de autenticación y autorización entre proveedores de servicios de identidad, proveedores de servicios y usuarios de los servicios. Permite el establecimiento de identidades portables, entendidas éstas como la extensión de las operaciones de autenticación y autorización de un dominio de seguridad a otro distinto con el que el primero mantiene una relación de confianza. La última versión es la 2.0 y data de 2005.

El uso más extendido de SAML es como mecanismo de Single Sign On y de federación de identidades¹ pero su alcance va más allá de la mera autenticación permitiendo intercambiar atributos de los usuarios de los servicios con el fin de establecer perfiles para procesos de autorización.

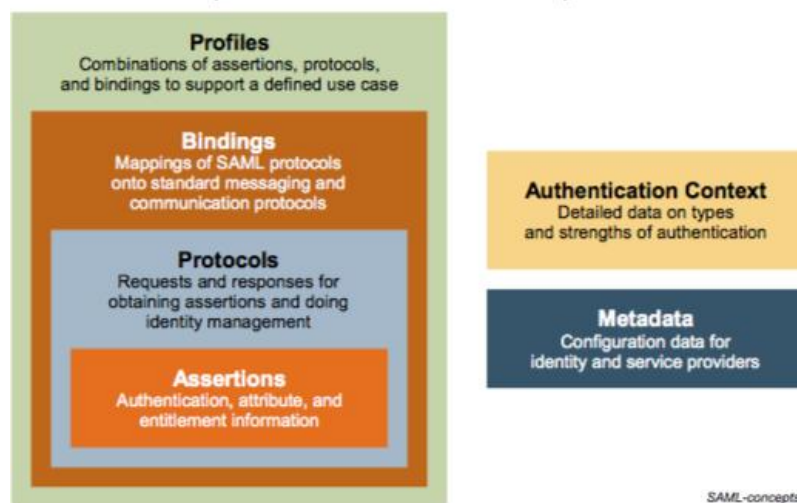
¹ Un sistema SSO es aquel en el que un usuario posee una identidad para distintos dominios de seguridad y no precisa volver a autenticarse para acceder a cualquiera de dichos dominios. Los sistemas SSO pueden disponer de un proveedor de identidad para todos los dominios o pueden ser federados. La Federación de identidades es un caso particular en el que cualquiera de los proveedores de servicio de la federación podría autenticar al usuario y éste no precisaría una nueva autenticación en el círculo de confianza. Dicho de otra forma: en una federación de identidades se establece un círculo de confianza que permite que un usuario autenticado en un organismo de la federación acceda a recursos de otro organismo de la misma federación sin autenticarse nuevamente.

SAML se usa intensiva y extensivamente en la Administración General del Estado en servicios como Cl@ve y en su extensión europea STORK.

Características generales

- **Múltiples tecnologías de transporte:** SAML es un lenguaje de marcado basado en XML por lo que en sí es independiente del mecanismo de transporte. Los mecanismos más comunes son HTTP (es el caso de Cl@ve), y SOAP. Se puede usar sólo uno de los medios de transporte o ambos, según el escenario.
- **Distribuido:** SAML no requiere una autoridad central. Basta con que el proveedor de servicio conozca alguna otra entidad de confianza que cuente con capacidad de autenticar al usuario para que solicite sus servicios o valide sus aserciones. No obstante, se puede usar para sistemas que cuentan con un único proveedor de servicios de identidad centralizado para un conjunto de dominios.

Mecanismos de funcionamiento



Conceptos básicos de SAML: perfiles, bindings, protocolo y aserciones (OASIS)

• Aserciones SAML

SAML define esquemas XML para expresar lo que denomina "aserciones" (assertions), que son verificaciones constatadas sobre un usuario. Existen 3 tipos:

- **Autenticaciones:** verificaciones de identidad. Cuando un proveedor de servicios de identidad recibe una solicitud de autenticación de un usuario lo primero que hace es pedir a dicho usuario sus credenciales de autenticación, que procesa de acuerdo a su política. Si el proceso de autenticación es satisfactorio, el proveedor declarará mediante una aserción de autenticación que el usuario X fue identificado a través del método M en el instante T y enviará un mensaje al proveedor de servicio que le solicitó la autenticación.

En concreto, creará un elemento *AuthenticationStatement*, que contendrá el nombre del usuario autenticado (*Subject*), el método (*AuthenticationMethod*) utilizado para la autenticación, y el instante (*AuthenticationInstant*). El proveedor de identidad puede usar múltiples mecanismos de autenticación: desde passwords a certificados X.509 pasando por tickets Kerberos.

La siguiente figura muestra una aserción de autenticación en la que un proveedor de identidad confirma que el usuario *joeuser* se ha autenticado mediante su *password* en un instante determinado.

```
<saml:Assertion ...>
  <saml:AuthenticationStatement
    AuthenticationMethod="password"
    AuthenticationInstant="2001-12-03T10:02:00Z">
    <saml:Subject>
      <saml:NameIdentifier
        SecurityDomain="smithco.com"
        Name="joeuser" />
      <saml:ConfirmationMethod>
        http://...core-25/sender-vouches
      </saml:ConfirmationMethod>
    </saml:Subject>
  </saml:AuthenticationStatement>
</saml:Assertion>
```

- **Autorizaciones:** mediante estas aserciones, las autoridades de autorización (normalmente las mismas que las de identificación) confirman o deniegan permisos de usuarios concretos ya identificados para realizar determinadas acciones. Las entidades autorizadas pueden ser usuarios humanos o sistemas.
- **Atributos.** Expresan datos sobre una entidad (normalmente personas) que permiten a los proveedores de servicio aplicar su propia política de control de acceso o de servicio en función de la respuesta. Por ejemplo, un comercio *online* podría solicitar a una entidad financiera datos sobre el límite de crédito de un usuario. La siguiente figura muestra una aserción de atributo en la que una entidad confirma que un usuario ha saldado su pago e informa de su límite de crédito.

```
<saml:Assertion ...>
  <saml:AttributeStatement>
    <saml:Subject>...</saml:Subject>
    <saml:Attribute
      AttributeName="PaidStatus"
      AttributeNamespace="http://smithco.com">
      <saml:AttributeValue>
        PaidUp
      </saml:AttributeValue>
    </saml:Attribute>
    <saml:Attribute
      AttributeName="CreditLimit"
      AttributeNamespace="http://smithco.com">
      <saml:AttributeValue>
        <my:amount currency="USD">500.00
        </my:amount>
      </saml:AttributeValue>
    </saml:Attribute>
  </saml:AttributeStatement>
</saml:Assertion>
```

- **Roles.** SAML considera 3 roles:
 - **Principal:** se denomina así a la entidad que requiere ser autenticada; puede ser un usuario final u otro sistema.
 - **Proveedor de servicio (SP):** desea autenticar, autorizar o conocer algún dato de algún usuario o sistema (principal)
 - **Proveedor de identidad (IDP):** autentica, autoriza o envía datos de usuarios (principales).

- **Protocolos**

SAML define esquemas XML para la implementación de protocolos sencillos Request/Response mediante los que se pueda solicitar aserciones y responderlas. Se trata de un esquema completamente separado del de aserciones.

Un requerimiento (request) contiene un claim mediante el que se solicita una aserción de autenticación, autorización o de atributos. La respuesta de la autoridad correspondiente contendrá la aserción resultante. Todo requerimiento debe contener el ID del sujeto en

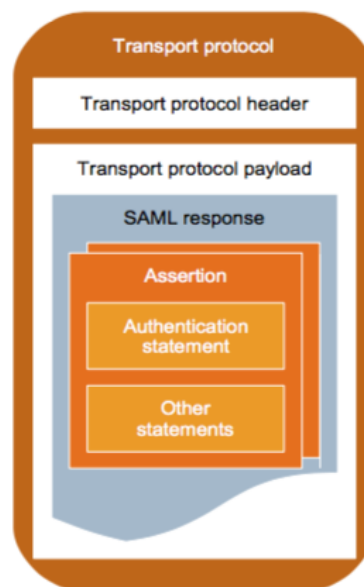
cuestión, que también se incluirá en la respuesta. Los requerimientos (Request) pueden ser, según se solicite, de autenticación, autorización o solicitud de atributos (*claim*).

La siguiente tabla muestra los protocolos y su función:

Protocolo	Función
Authentication Request Protocol	Solicitud de autenticación de un usuario
Single Logout Protocol	Define la forma de realizar un logout simultáneo de las sesiones asociadas a un usuario. El logout puede ser iniciado directamente por el usuario, o iniciado por un IDP o SP debido a la expiración por tiempo o también por decisión de un administrador
Assertion Query and Request Protocol	Solicitud de aserciones. La forma de Request de este protocolo permite solicitar a un IDP un Assertion haciendo referencia a su ID. La forma de Query define como solicitar Assertions en base al usuario y el tipo de Assertion
Name Identifier Management Protocol	Provee mecanismos para cambiar el valor o formato del identificador usado para referirse al usuario. El protocolo también define la forma de terminar una asociación de la identidad del usuario entre el IDP y el SP
Name Identifier Mapping Protocol	Provee un mecanismo para poder mapear un identificador de usuario usado entre un IDP y un SP de forma de obtener el utilizado entre el IDP y otro SP.
Artifact Resolution Protocol	Permite transportar los Assertions por referencia (a esta referencia se le denomina Artifact en el estándar) en vez de valor. Esto facilita el uso de SAML en un entorno web.

- **Bindings**

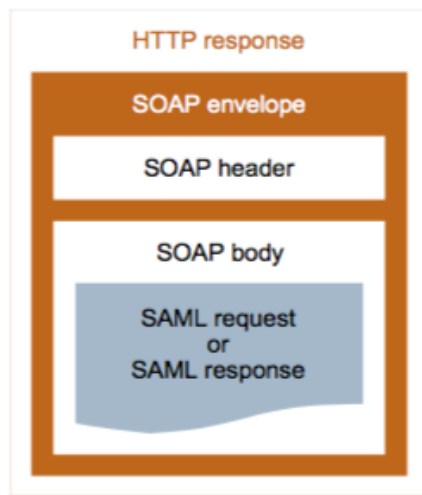
Un *binding* es especificación que indica cómo encapsular los mensajes Request/Response del protocolo SAML en el payload del mensaje de un mecanismo de transporte concreto.



Encapsulado de un mensaje SAML en el payload de un protocolo de transporte genérico.

SAML define numerosos bindings. Algunos de los más importantes son:

- Binding de SAML sobre SOAP (**Binding SOAP**): indica cómo insertar un mensaje SAML en el cuerpo (*body*) de un mensaje SOAP con indicaciones de cómo transportar a su vez el mensaje SOAP sobre HTTP. No se debe confundir con el envío de aserciones en la cabecera (*header*) de los mensajes SOAP definido por WS-Security que tienen como función proporcionar seguridad al mensaje SOAP en sí y no ofrecer un servicio de autenticación o autorización.



SAML sobre SOAP

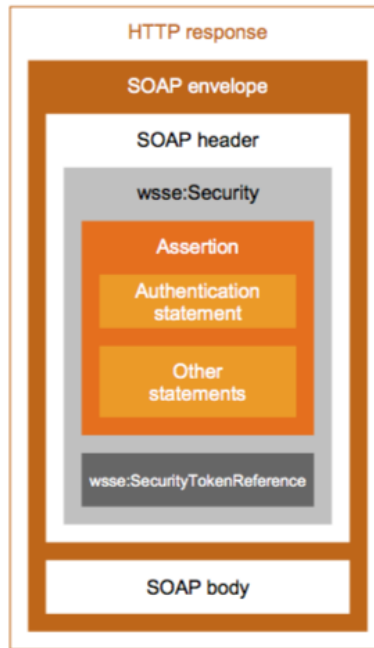
- Binding de SAML sobre el método POST de HTTP. (**Binding POST**)
- Binding de SAML sobre mensajes HTTP de redirección (**Binding Redirección**).
- **Perfiles SAML**
 Los perfiles SAML definen cómo combinar aserciones SAML, protocolos y bindings para poder cubrir determinados escenarios de uso y garantizar la interoperabilidad.
 Cabe destacar:
 - Perfil SSO para browsers (Web Browser SSO Profile). Pensado para ofrecer servicios de SSO a clientes con navegadores estándar.
 - Perfil mejorado de cliente y proxy (Enhanced Client and Proxy - ECP). Caso especial en que se emplea un mediador de proveedores de servicios de identidad (caso de Cl@ve).

SAML y Web Service-Security

Como se ha comentado, WSS consiste en un conjunto de especificaciones para proporcionar confidencialidad, integridad y autenticación a mensajes SOAP. Para ello se especifica un tag (*wsse*) para la cabecera de los mensajes SOAP dentro del cual se coloca la información que proporciona la seguridad al mensaje.

Al contrario que la especificación de SAML sobre SOAP, en que los mensajes del protocolo SAML se incluyen en el cuerpo del mensaje SOAP y son parte de un protocolo de autenticación y autorización, en la norma WSS sólo se especifica como insertar aserciones SAML encaminadas a proteger el mensaje SOAP en que se encuentran. Típicamente contienen claves empleadas para firmar los mensajes que una vez verificadas permiten confirmar la identidad del emisor del mensaje SOAP.

Es decir, mientras que los mensajes de SAML sobre SOAP van en el cuerpo (*body*) de SOAP y forman parte de un protocolo Request/Response de autenticación, autorización o consulta de atributos, los mensajes SAML de la norma WSS son sólo aserciones que van en la cabecera (*header*) SOAP y tienen como objetivo participar de la seguridad del mensaje SOAP en sí. Normalmente son aserciones de autenticación que van firmadas mediante XML DSiganture y que permiten establecer la identidad del emisor del mensaje SOAP.



Mensaje SOAP con extensiones WSS que incluye aserciones SAML como tokens de seguridad

4.5 JSON Web Token

JSON Web Token (JWT) es un estándar abierto (RFC 7519) que define una forma compacta y autónoma para transmitir de forma segura información entre partes como un objeto JSON. Esta información puede ser verificada y confiable porque está firmada digitalmente. Los JWT se pueden firmar usando un secreto (con el algoritmo HMAC) o un par de claves públicas/privadas usando RSA o ECDSA.

Aunque los JWT se pueden cifrar para proporcionar también secreto entre las partes, nos centraremos en los tokens firmados. Los tokens firmados pueden verificar la integridad de las reclamaciones contenidas en él, mientras que los tokens cifrados ocultan esas reclamaciones a otras partes. Cuando los tokens se firman utilizando pares de claves públicas / privadas, la firma también certifica que solo la parte que tiene la clave privada es la que la firmó (no repudio).

Casos de uso

- **Autorización:** Este es el escenario más común para usar JWT. Una vez que el usuario ha iniciado sesión, cada solicitud posterior incluirá el JWT, permitiendo al usuario acceder a rutas, servicios y recursos que están permitidos con ese token. Single Sign On es una característica que utiliza ampliamente JWT hoy en día debido a su pequeña sobrecarga y su capacidad para ser utilizado fácilmente en diferentes dominios.
- **Intercambio de información:** los tokens web JSON son una buena manera de transmitir información de forma segura entre las partes. Debido a que los JWT se pueden firmar, por ejemplo, utilizando pares de claves públicas / privadas, puede estar seguro de que los remitentes son quienes dicen que son. Además, como la firma se calcula utilizando el encabezado y la carga útil, también puede verificar que el contenido no se haya manipulado.

Estructura

- **Cabecera:** El encabezado típicamente consiste en dos partes: el tipo de token, que es JWT, y el algoritmo de firma que se está utilizando, como HMAC SHA256 o RSA.
- **Carga útil:** La segunda parte del token es la carga útil, que contiene las reclamaciones (claims). Las reclamaciones son declaraciones sobre una entidad (normalmente, el

usuario) y datos adicionales. Hay tres tipos de reclamaciones: reclamaciones registradas, públicas y privadas.

- Reclamaciones registradas: Se trata de un conjunto de reivindicaciones predefinidas que no son obligatorias pero recomendadas, para proporcionar un conjunto de reclamaciones útiles e interoperables. Algunos de ellos son: iss (emisor), exp (tiempo de caducidad), sub (sujeto), aud (audiencia) y otros.
- Reclamaciones públicas: Estas pueden definirse a voluntad por aquellos que usan JWT. Pero para evitar colisiones, deben definirse en el Registro de Tokens Web de IANA JSON o definirse como un URI que contiene un espacio de nombres resistente a las colisiones.
- Reclamaciones privadas: Estas son las reclamaciones personalizadas creadas para compartir información entre las partes que acuerdan su uso y no son reclamaciones registradas o públicas.
- Firma: Para crear la parte de firma, debe tomar el encabezado codificado, la carga útil codificada, un secreto, el algoritmo especificado en el encabezado y firmarlo.

En su forma compacta, los tokens web JSON consisten en tres cadenas Base64-URL separadas por puntos (.), que son:

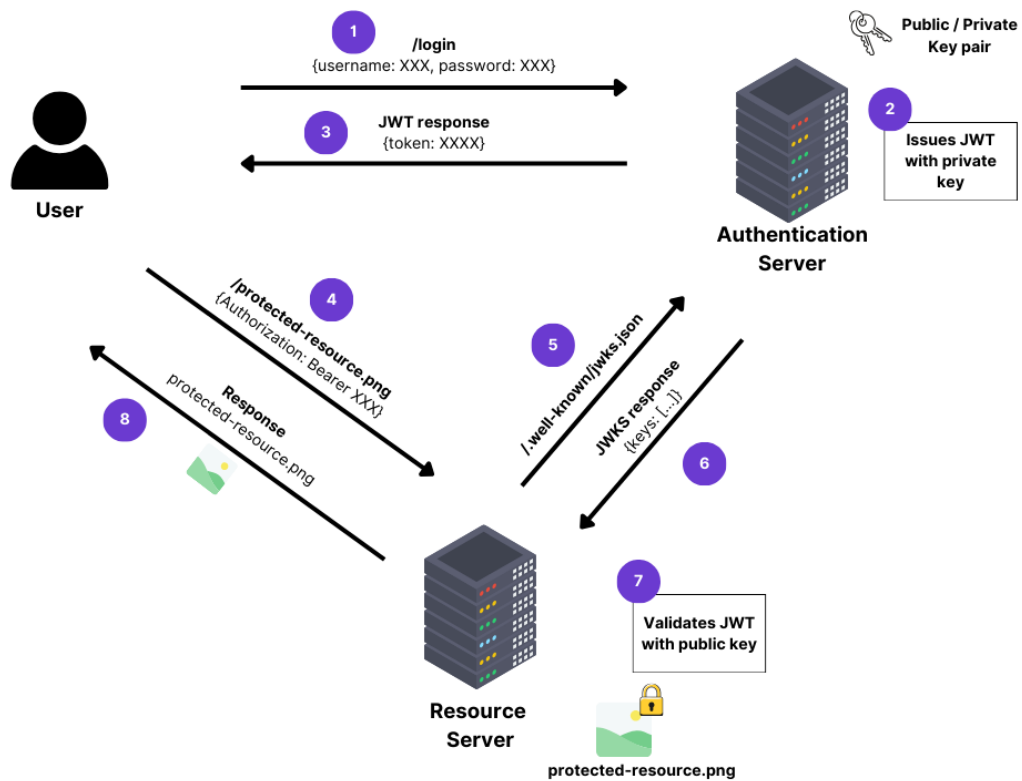
xxxxx.yyyyy.zzzzz
<header(cabecera).Payload(Carga útil).Signature(firma)>

Funcionamiento

Los JWT se emiten y verifican. Cuando se emite, un servidor genera un nuevo JWT generalmente después de un proceso de inicio de sesión. Este paso implica añadir toda la información del usuario como reclamaciones JWT y firmar los datos con una clave privada. El cliente de usuario almacena este token y lo adjunta a cada solicitud de API futura.

Comúnmente, los JWT se envían como tokens de portador en el encabezado de autorización HTTP, un método popularizado por la especificación OAuth 2.0 (RFC 6750). Esto implica incluir el JWT en el encabezado de autorización de solicitudes HTTP con el prefijo 'Bearer' (portador). El término "bearer" indica que al poseedor del token se le concede acceso al recurso protegido.

El servidor que recibe esta solicitud puede validar la firma del token con la clave pública y luego desempaquetar su contenido y autorizar la llamada. La clave pública es un elemento esencial por lo que los servidores deben acceder a esta clave de alguna manera.



<https://www.nblocks.dev/blog/authentication/introduction-to-json-web-tokens-jwt>

En esta figura el usuario quiere recuperar un recurso protegido del “Servidor de recursos”. Después de iniciar sesión y obtener un JWT desde el “Servidor de autenticación”, el usuario puede proporcionar este token al “Servidor de recursos” que tomará la clave pública del “Servidor de autenticación” y luego renderizará el recurso protegido.

Habitualmente, el login inicial se ha redireccionado desde el proxy inverso o API Gateway del sistema cuyos recursos se quieren acceder. Se debe tener en cuenta que con el término usuario puede tratarse de una persona (presentando sus credenciales como usuario y contraseña, certificado digital o otros medios autorizados) o un servicio web (identificándose por ejemplo mediante certificado cualificado de sitio web, acorde al Reglamento eIDAS).

5 Seguridad de las bases de datos

Otro de los elementos críticos de un sistema, son sus repositorios de datos, que generalmente se implementan a través de bases de datos relacionales, documentales o de otro tipo. Las vulnerabilidades más habituales que nos encontramos en las bases de datos son las siguientes:

- **Debilidad de credenciales:** Un usuario o una contraseña que puedan ser capturados por un atacante para acceder directamente al repositorio de datos impersonando un usuario válido.
- **Gestión de usuarios mediante grupos:** Una buena práctica es que los usuarios de una base de datos reciban sus privilegios a través de grupos, de forma que la seguridad se pueda manejar de forma colectiva.
- **Funciones no utilizadas:** Es conveniente, tanto en la base de datos, como en otro tipo de software asegurarnos que únicamente instalamos el software mínimo necesario para la operativa que tenemos. Una buena forma de evitar incorporar bugs a un sistema es no desplegar funcionalidades que no vamos a utilizar.
- **Mínimos privilegios:** Es otra buena práctica el definir para un grupo de usuarios los mínimos privilegios necesarios que precisa para operar en el sistema. No deben utilizarse más que cuando no exista otra alternativa para los usuarios administradores de la base de datos.
- **Elección y mantenimiento adecuados del SGBD:** Una elección del sistema de base de datos adecuada y un correcto y puntual mantenimiento del mismo pueden evitar problemas en la operación del mismo y riesgos de seguridad.
- **Inyección SQL:** Uno de los principales riesgos de uso de un gestor de base de datos, en particular cuando utilizamos bases de datos relacionales, es la inyección SQL. Una buena opción para protegerse del mismo es validar todas las entradas que se remiten al gestor de base de datos.

